

# The Dining Philosophers Problem

A classic synchronization problem about deadlock and starvation

## Definition

The **dining philosophers problem** is a classic synchronization problem introduced by Dijkstra in 1965. It asks how several competing processes can share limited resources without deadlock or starvation.

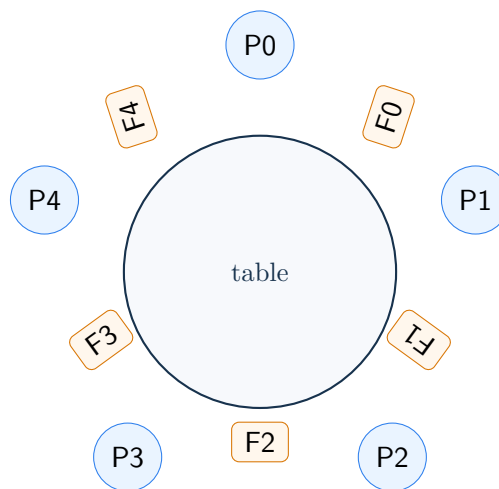
## 1. Problem Setup

### Dining Table

Five philosophers sit around a circular table. Each philosopher has a plate of spaghetti, and there is one fork between each pair of neighboring plates.

### Two-Fork Requirement

A philosopher needs two forks to eat: the fork on the left and the fork on the right. A philosopher can eat only after acquiring both forks.



## 2. Philosopher Behavior

### Life Cycle

Each philosopher repeatedly alternates between thinking and eating.

```
while true:
    think
    acquire left fork
    acquire right fork
    eat
    release left fork
    release right fork
```

### Main Question

Can we write a program for each philosopher so every philosopher can eat when possible and the system never gets stuck?

## 3. Why the Problem Matters

### Synchronization Model

The philosophers represent processes. The forks represent shared resources. Eating represents entering a critical activity that requires multiple resources.

### Core Difficulty

The difficult part is not taking one fork. The difficult part is coordinating two shared resources without creating deadlock or starvation.

## 4. The Obvious Nonsolution

### Take Left, Then Take Right

The most direct solution is for each philosopher to take the left fork, then the right fork, then eat.

```
#define N 5

void philosopher(int i)
{
    while (TRUE) {
        think();
        take_fork(i);
        take_fork((i + 1) % N);
        eat();
        put_fork(i);
        put_fork((i + 1) % N);
    }
}
```

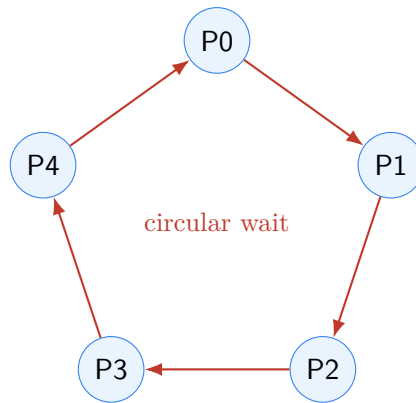
### Why It Fails

If all five philosophers take their left forks at the same time, every philosopher holds one fork and waits forever for the right fork. This is deadlock.

## 5. Deadlock

### Deadlock

**Deadlock** occurs when every process in a set is waiting for an event that only another process in the same set can cause.



### Dining Philosophers Deadlock

Each philosopher holds the left fork and waits for the right fork. Since nobody releases a fork, nobody can continue.

## 6. Try-and-Retry Attempt

### Modified Idea

After taking the left fork, a philosopher checks whether the right fork is available. If not, the philosopher puts down the left fork, waits, and tries again.

### Still Not Enough

If all philosophers act in lockstep, they may all pick up their left forks, all fail to get their right forks, all put down their left forks, and all repeat forever.

## 7. Starvation

### Starvation

**Starvation** occurs when processes keep running or retrying, but some process never makes progress.

### Difference from Deadlock

In deadlock, the processes are stuck waiting. In starvation, activity may continue, but useful progress for some process never occurs.

### Dining Philosophers Starvation

The philosophers may repeatedly release and retry without anyone actually eating.

## 8. Random Waiting

### Random Backoff

One practical improvement is to wait for a random amount of time before trying again. This reduces the chance that all philosophers remain synchronized forever.

### Practical Use

Random retry works well in many systems. For example, network protocols can use random waiting after collisions.

### Not a Proof

Random waiting makes failure unlikely, but it does not provide a deterministic guarantee. Some safety-critical systems require a solution that always works.

## 9. Single Mutex Solution

### Simple Safe Idea

Protect the fork-acquisition and fork-release code with one binary semaphore, or mutex. Only one philosopher can try to acquire forks at a time.

```
while true:
    think
    down(mutex)
    take left fork
    take right fork
    eat
    put left fork
    put right fork
    up(mutex)
```

### Correctness

This avoids deadlock because only one philosopher can hold forks while entering the eating section.

### Performance Bug

Only one philosopher can eat at a time. With five forks, two non-neighboring philosophers should be able to eat simultaneously.

## 10. Better Semaphore Solution

### State-Based Solution

A better solution keeps a state for each philosopher: thinking, hungry, or eating. A philosopher may eat only if neither neighbor is eating.

| State           | Meaning                                                      |
|-----------------|--------------------------------------------------------------|
| <b>THINKING</b> | The philosopher is not trying to eat.                        |
| <b>HUNGRY</b>   | The philosopher wants to eat and is trying to acquire forks. |
| <b>EATING</b>   | The philosopher has both forks and is eating.                |

#### Neighbor Rule

Philosopher  $i$  can enter EATING only if philosopher  $i$ 's left neighbor and right neighbor are not eating.

## 11. Neighbor Macros

#### Circular Table

With  $N$  philosophers numbered 0 to  $N - 1$ , the neighbors of philosopher  $i$  are:

$$\text{LEFT} = (i + N - 1) \bmod N$$

$$\text{RIGHT} = (i + 1) \bmod N$$

#### Example

If  $N = 5$  and  $i = 2$ , then LEFT is 1 and RIGHT is 3.

## 12. Data Structures

| Object               | Purpose                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------|
| <i>state</i> [ $N$ ] | Records whether each philosopher is THINKING, HUNGRY, or EATING.                            |
| <i>mutex</i>         | Protects updates to the shared state array.                                                 |
| <i>s</i> [ $N$ ]     | One semaphore per philosopher, used to block hungry philosophers until forks are available. |

#### Key Design

The mutex protects the shared bookkeeping. The per-philosopher semaphores allow individual hungry philosophers to sleep until they are allowed to eat.

## 13. Core Algorithm

```
#define N 5
#define LEFT (i + N - 1) % N
#define RIGHT (i + 1) % N

#define THINKING 0
#define HUNGRY 1
```

```

#define EATING 2

int state[N];
semaphore mutex = 1;
semaphore s[N];

void philosopher(int i)
{
    while (TRUE) {
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

```

## 14. Taking Forks

```

void take_forks(int i)
{
    down(&mutex);
    state[i] = HUNGRY;
    test(i);
    up(&mutex);
    down(&s[i]);
}

```

### Meaning

The philosopher marks itself HUNGRY, tests whether both forks are available, then blocks on its own semaphore if it cannot eat yet.

## 15. Putting Forks

```

void put_forks(int i)
{
    down(&mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT);
    up(&mutex);
}

```

### Meaning

After eating, the philosopher becomes THINKING and checks whether either neighbor can now start eating.

## 16. The Test Procedure

```

void test(int i)
{
    if (state[i] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}

```

### Meaning

The test procedure grants permission to eat only when philosopher  $i$  is hungry and neither neighbor is eating.

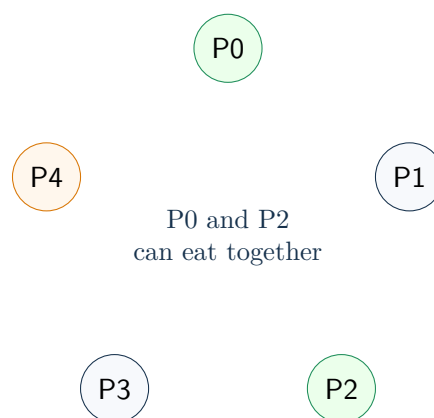
## 17. Why the Better Solution Works

| Property                 | Reason                                                                                                |
|--------------------------|-------------------------------------------------------------------------------------------------------|
| No neighboring eaters    | A philosopher can eat only if both neighbors are not eating.                                          |
| No deadlock              | Hungry philosophers do not hold one fork while waiting for another. They wait on their own semaphore. |
| More parallelism         | Non-neighboring philosophers can eat at the same time.                                                |
| Controlled state changes | Shared state is updated while holding mutex.                                                          |

### Maximum Parallelism for Five

With five philosophers, at most two philosophers can eat at the same time, because any third philosopher would be adjacent to one of them.

## 18. Parallel Eating Example



### Observation

P0 and P2 are not neighbors, so they can eat at the same time. P4 cannot eat while P0 is eating because P4 is P0's neighbor.

## 19. Comparison of Attempts

| Attempt                      | Main Idea                                        | Problem                                       |
|------------------------------|--------------------------------------------------|-----------------------------------------------|
| <b>Take left, then right</b> | Simple fork acquisition.                         | Can deadlock.                                 |
| <b>Release and retry</b>     | Put down left fork if right fork is unavailable. | Can starve in lockstep.                       |
| <b>Random retry</b>          | Wait random time before trying again.            | Practical, but not a deterministic guarantee. |
| <b>Single mutex</b>          | Only one philosopher enters fork code at a time. | Safe but allows only one eater.               |
| <b>State + semaphores</b>    | Eat only when neither neighbor is eating.        | More complex, but gives safe parallelism.     |

## 20. Big Picture

### Main Conclusion

The dining philosophers problem shows that sharing multiple resources is harder than protecting one critical section. A correct solution must avoid deadlock, avoid starvation, and still allow as much safe parallelism as possible.

### Exam Shortcut

Remember: taking the left fork then the right fork can deadlock. Releasing and retrying can starve. A single mutex is safe but too restrictive. The state-array semaphore solution lets a philosopher eat only when neither neighbor is eating.



## 21. Quick Review

### Key Points

- Dijkstra introduced the dining philosophers problem in 1965.
- Five philosophers share five forks around a circular table.
- Each philosopher needs both left and right forks to eat.
- Taking the left fork and then the right fork can deadlock.
- Deadlock occurs when everyone waits forever for someone else to release a resource.
- Releasing the left fork and retrying can still cause starvation.
- Random waiting is useful in practice but does not guarantee correctness.
- A single mutex prevents deadlock but allows only one philosopher to eat at a time.
- A better solution tracks THINKING, HUNGRY, and EATING states.
- A philosopher may eat only if neither neighbor is eating.
- One semaphore per philosopher lets hungry philosophers block individually.
- The mutex protects the shared state array.